Roguelike RPG Dungeon: Design Document

A

This document outlines the architectural decisions and design choices for the Roguelike RPG Dungeon overhaul.

Class Diagram

The system is designed with a strong emphasis on modularity, scalability, and flexibility. Core gameplay mechanics are abstracted through interfaces and base classes, allowing new features to be added without modifying existing systems.

Key design decisions include:

Loose coupling through interfaces (IDamageable, IConsumable) to reduce dependency between systems
Composition over inheritance for dynamic stat calculation and equipment management
Separation of concerns between gameplay logic (RPGGame), progression (DungeonManager), and economy (Shop)

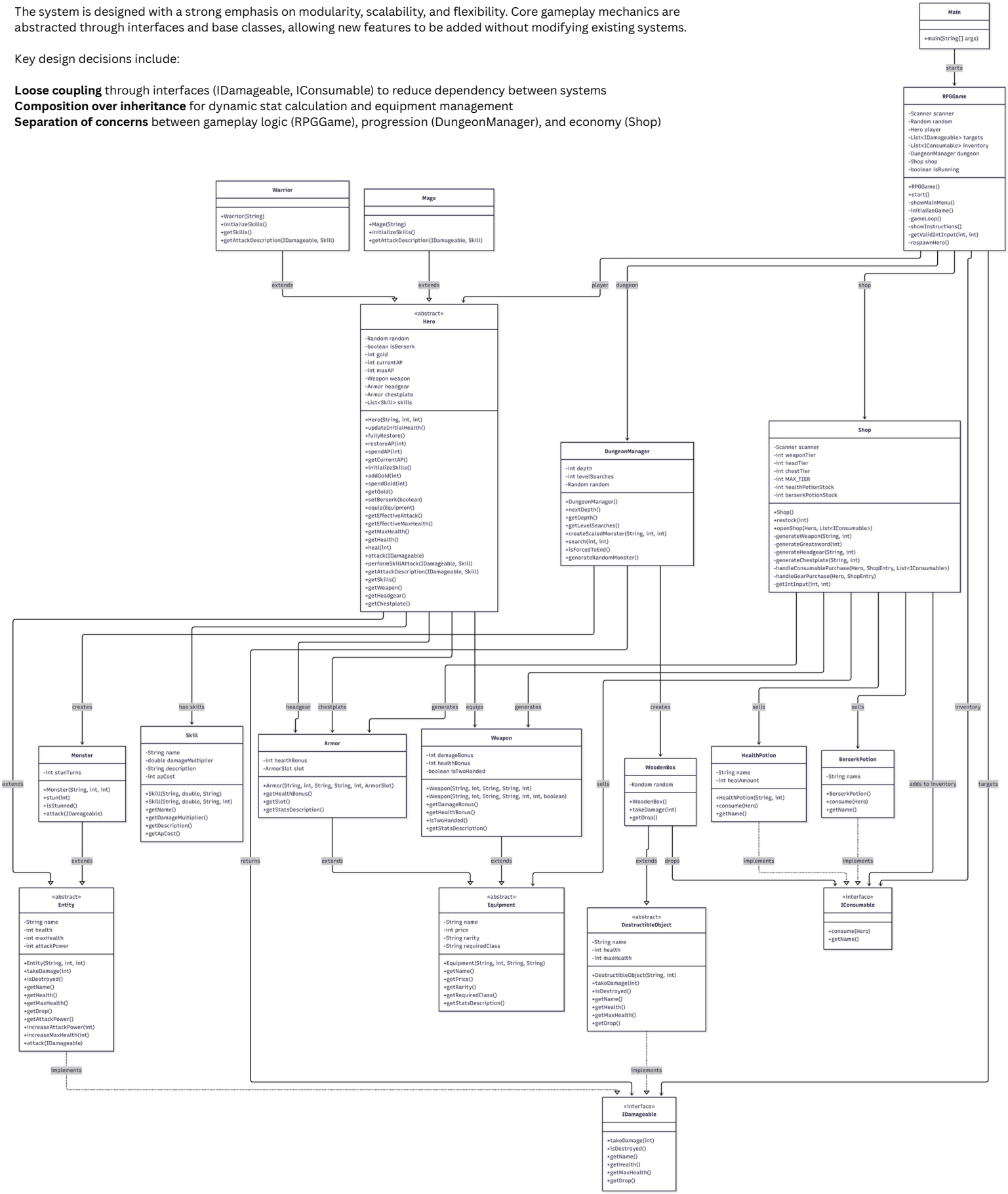

Figure 1 Full class diagram that shows relationship between class in the program

Roguelike RPG Dungeon: Design Document

B

This document outlines the architectural decisions and design choices for the Roguelike RPG Dungeon overhaul.

Flow Diagram

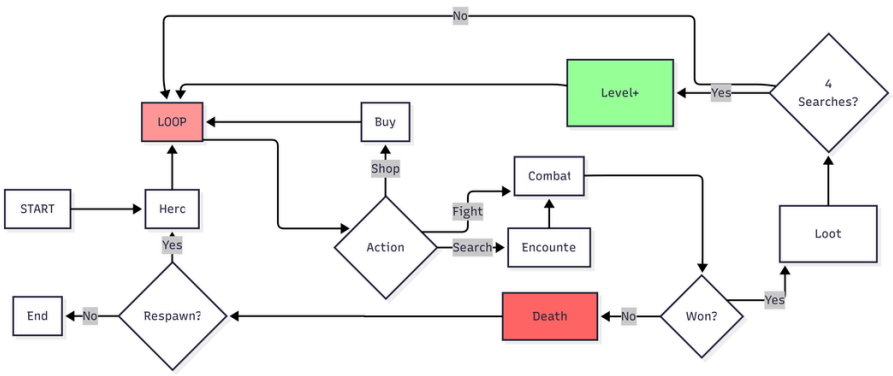

Figure 2 Full action diagram that shows player journey flowchart in the program

Design Rationale

1. Roguelike Progression & Scaling

- **DungeonManager** : Centralizes depth tracking and enemy scaling logic. Stats for `Monster` objects scale linearly with depth, ensuring a consistent challenge as the player progresses.
- **Search Mechanism** : Instead of static encounters, the "Search" mechanic introduces randomness and choice, key pillars of the Roguelike genre.

2. Equipment & Stat System

- **Composition over Inheritance** : The `Hero` class uses composition to manage its gear (`Weapon`, `Armor`). This allows for dynamic stat calculation where the effective attack and health are derived from base stats plus equipment bonuses.
- **Encapsulation** : Equipment details (damage bonuses, health bonuses) are encapsulated within their respective classes, following the Single Responsibility Principle.

3. Combat Skills & Strategy

- **Skill Class** : Instead of a single "Attack" method, heroes now have a `List<Skill>`. This provides depth to combat through an AP (Action Point) system, where players must manage energy costs to choose between standard hits (which restore AP) and high-damage (but costly) special moves.
- **Polymorphism** : The performSkillAttack method in Hero uses the provided Skill object to calculate final damage, allowing different skills to behave uniquely without complex conditional logic.

4. Meta-Progression & Persistence

- **State Preservation** : The `respawnHero` method in `RPGGame` demonstrates how object state (Gold, Gear) can be saved and transferred to a new instance of a `Hero` after death, creating a "Roguelite" loop.
- **Currency System** : The introduction of Gold and a Shop provides long-term goals and a way to mitigate RNG through strategic purchases.
- **Random Dungeon Events** : The "Search" mechanic in `DungeonManager` introduces dynamic gameplay events such as `DANGER_ZONE` (monster buffs), `FOGGY_AREA` (multiple spawns), and `TRAP` (percentage damage), which add strategic risk-assessment to exploration.
- **Integrated Shop System** : The introduction of Gold and a tiered `Shop` provides long-term progression, allowing players to upgrade gear from Tier 1 to 5 to keep pace with scaling enemies.

5. OOP Criteria Fulfillment

- **Abstraction** : `IDamageable` and `IConsumable` interfaces allow the game engine to interact with objects without knowing their concrete implementation.
- **Inheritance** : A rich hierarchy from `Entity` to `Warrior`/`Mage` and `Equipment` to `Weapon`/`Armor` ensures code reuse and logical structuring.
- **Polymorphism** : Used extensively in combat (skills), drops (boxes/monsters), and items (potions).
- **Encapsulation** : Strict use of access modifiers and public API methods (e.g., `increaseMaxHealth`) to protect internal object state.